

ADA - Análisis y Diseño de Algoritmos (2026-1)

Tarea 2

Nombre: Miguel Angel Padilla Rosero
Código: 8988878
Profesores: Carlos Ramirez y Camilo Rocha
Fecha: 10 de Marzo de 2026

Índice

1. Problemas conceptuales	2
1.1. Lightweight consulting business (Ej. 6-4, Kleinberg & Tardos, p. 315)	2
1.2. Forward and backward substring (Ej. 3.8, Erickson, p. 128)	13
2. Problemas prácticos (entregados en la arena)	19

1. Problemas conceptuales

1.1. Lightweight consulting business (Ej. 6-4, Kleinberg & Tardos, p. 315)

Resumen del problema

Suponga que se está administrando un pequeño negocio de consultoría: solo usted, dos asociados y algunos equipos rentados. Los clientes están distribuidos entre la Costa Este y la Costa Oeste de Estados Unidos.

Cada mes, se puede operar el negocio desde una oficina en New York (NY) o desde una oficina en San Francisco (SF). En el mes i , se incurre en un costo operativo N_i si el negocio opera desde NY, y en un costo operativo S_i si opera desde SF.

Además, si en el mes i se opera desde una ciudad y en el mes $i + 1$ desde la otra, entonces se incurre en un costo fijo de mudanza M .

Dado un número de meses n , el costo de mudanza M , y las secuencias de costos operativos

$$N_1, \dots, N_n \quad \text{y} \quad S_1, \dots, S_n,$$

se debe encontrar un plan de costo mínimo.

Ejemplo del enunciado

Suponga que $n = 4$, $M = 10$, y que los costos operativos son:

	Mes 1	Mes 2	Mes 3	Mes 4
NY	1	3	20	30
SF	50	20	2	4

Entonces, el plan de costo mínimo es:

$$[\text{NY}, \text{NY}, \text{SF}, \text{SF}]$$

con costo total

$$1 + 3 + 2 + 4 + 10 = 20.$$

Preguntas

- (a) Muestre que el siguiente algoritmo **no** resuelve correctamente el problema, dando una instancia en la que no devuelve la respuesta correcta:

for $i = 1$ to n
 if $N_i < S_i$ then output “NY in month i ”
 else output “SF in month i ”

En su ejemplo, diga cuál es la respuesta correcta y también cuál es la respuesta que encuentra el algoritmo.

- (b) Dé un ejemplo de una instancia en la cual todo plan óptimo deba moverse al menos tres veces. Proporcione una breve explicación de por qué su ejemplo tiene esta propiedad.
- (c) Dé un algoritmo eficiente que tome como entrada los valores de n , M , y las secuencias de costos $N_1, \dots, N_n$ y $S_1, \dots, S_n$, y retorne el **costo** de un plan óptimo.

(a) El algoritmo codicioso no es correcto

Resumen del problema

Se debe mostrar que el algoritmo propuesto no siempre encuentra un plan óptimo. Para ello, basta construir una instancia en la que escoger, mes a mes, la ciudad con menor costo operativo produzca una solución peor que otra que considere el costo de mudanza.

Idea del contraejemplo

La idea es construir un ejemplo en el que cambiar de ciudad parezca conveniente si solo se comparan los costos mes a mes, pero donde dicho cambio en realidad no compensa el costo fijo de mudanza M .

Instancia propuesta

Tomemos:

$$n = 5, \quad M = 20.$$

Y los costos operativos:

	M_1	M_2	M_3	M_4	M_5
NY	1	2	2	8	10
SF	5	6	8	1	5

Plan óptimo correcto

El plan de costo mínimo es:

$$[\text{NY}, \text{NY}, \text{NY}, \text{NY}, \text{NY}].$$

Su costo total es:

$$1 + 2 + 2 + 8 + 10 = 23.$$

Plan que encuentra el algoritmo dado

El algoritmo compara mes a mes los costos y escoge la ciudad más barata en cada caso:

$$\begin{aligned}
M_1 &: \text{NY} & (1 < 5) \\
M_2 &: \text{NY} & (2 < 6) \\
M_3 &: \text{NY} & (2 < 8) \\
M_4 &: \text{SF} & (1 < 8) \\
M_5 &: \text{SF} & (5 < 10)
\end{aligned}$$

Por lo tanto, el algoritmo devuelve el plan:

$$[\text{NY}, \text{NY}, \text{NY}, \text{SF}, \text{SF}].$$

Costo del plan encontrado por el algoritmo

El costo operativo de este plan es:

$$1 + 2 + 2 + 1 + 5 = 11.$$

Como hay un cambio de ciudad entre M_3 y M_4 , se suma además el costo de mudanza $M = 20$. Entonces, el costo total es:

$$1 + 2 + 2 + 1 + 5 + 20 = 31.$$

Conclusión

El algoritmo propuesto devuelve el plan

$$[\text{NY}, \text{NY}, \text{NY}, \text{SF}, \text{SF}]$$

con costo total 31, mientras que el verdadero plan óptimo es

$$[\text{NY}, \text{NY}, \text{NY}, \text{NY}, \text{NY}]$$

con costo total 23.

Por lo tanto, el algoritmo **no es correcto**, ya que escoger en cada mes la ciudad con menor costo operativo no garantiza minimizar el costo total del plan.

(b) Instancia donde todo plan óptimo se mueve al menos tres veces

Resumen del problema

Se debe dar un ejemplo de una instancia en la cual *todo* plan óptimo tenga que cambiar de ciudad al menos tres veces. La idea es construir los costos de forma que, en varios meses, una ciudad sea considerablemente más barata que la otra, de modo que evitar un cambio de ciudad resulte más costoso que pagar la mudanza.

Idea del ejemplo

Tomamos una secuencia de 5 meses en la que la ciudad más barata va alternando entre SF y NY. Además, escogemos un costo de mudanza M suficientemente pequeño frente a la diferencia de costos operativos en los meses clave, para que convenga realizar los cambios de ciudad.

Instancia propuesta

Tomemos:

$$n = 5, \quad M = 15.$$

Y los costos operativos:

	M_1	M_2	M_3	M_4	M_5
NY	50	30	5	40	10
SF	10	20	40	20	45

Plan óptimo

Un plan de costo mínimo es:

$$[\text{SF}, \text{SF}, \text{NY}, \text{SF}, \text{NY}].$$

Este plan cambia de ciudad tres veces:

$$\text{SF} \rightarrow \text{NY}, \quad \text{NY} \rightarrow \text{SF}, \quad \text{SF} \rightarrow \text{NY}.$$

Costo del plan

El costo operativo de este plan es:

$$10 + 20 + 5 + 20 + 10 = 65.$$

Como hay tres cambios de ciudad y cada uno cuesta $M = 15$, el costo total de mudanza es:

$$3 \cdot 15 = 45.$$

Por lo tanto, el costo total del plan es:

$$10 + 20 + 5 + 20 + 10 + 15 + 15 + 15 = 110.$$

Justificación

En esta instancia, en los meses 1 y 2 conviene operar desde SF, porque sus costos (10 y 20) son menores que los de NY (50 y 30). En el mes 3 conviene cambiar a NY, porque su costo es 5, mientras que en SF es 40. En el mes 4 vuelve a convenir SF, porque cuesta 20 frente a 40 en NY. Finalmente, en el mes 5 conviene nuevamente NY, porque cuesta 10 frente a 45 en SF.

Si se evita alguno de estos cambios, el aumento en costos operativos supera el ahorro de la mudanza. Por eso, cualquier plan que intente reducir el número de cambios termina teniendo un costo mayor.

Conclusión

La instancia anterior tiene la propiedad de que un plan óptimo debe cambiar de ciudad al menos tres veces. Por lo tanto, constituye un ejemplo válido para el inciso.

(c) Algoritmo eficiente para hallar el costo óptimo

Resumen del problema

En este inciso se pide diseñar un algoritmo eficiente que, dados el número de meses n , el costo de mudanza M , y los arreglos de costos operativos

$$NY[1..n] \quad \text{y} \quad SF[1..n],$$

retorne el **costo** de un plan óptimo. La dificultad principal del problema está en que la decisión tomada en un mes afecta el costo total de los meses siguientes, ya que cambiar de ciudad implica pagar un costo fijo de mudanza.

Especificación del problema

Entrada:

- Un entero $n \in \mathbb{N}$, con $n \geq 1$, que representa el número total de meses.
- Un entero $M \in \mathbb{N}$, con $M \geq 0$, que representa el costo fijo de mudarse de una ciudad a la otra.
- Dos arreglos de enteros

$$NY[1..n] \quad \text{y} \quad SF[1..n],$$

donde, para todo $1 \leq i \leq n$:

- $NY[i]$ es el costo operativo de trabajar en New York durante el mes i ;
- $SF[i]$ es el costo operativo de trabajar en San Francisco durante el mes i .

Salida:

Un entero

$$OPT[n],$$

tal que $OPT[n]$ representa el costo mínimo posible de un plan válido de operación durante los n meses.

Estrategia de Programación Dinámica a utilizar

Se utilizará **programación dinámica con memorización (top-down)**. La idea es definir una única función recursiva

$$\phi(i, isNewYork),$$

donde:

- i representa el número de meses considerados;
- $isNewYork$ es un valor booleano que indica la ciudad en la que termina el plan en el mes i .

Más exactamente:

- si $isNewYork = \text{true}$, entonces el estado representa el costo mínimo de un plan válido para los primeros i meses que termina en New York;
- si $isNewYork = \text{false}$, entonces el estado representa el costo mínimo de un plan válido para los primeros i meses que termina en San Francisco.

La elección de esta estrategia se justifica porque:

- el problema tiene **subestructura óptima**, ya que la mejor decisión para el mes i depende de las mejores decisiones para el mes $i - 1$;
- existen **subproblemas superpuestos**, pues los mismos estados recursivos vuelven a aparecer en múltiples llamadas.

Planteamiento de la función objetivo

Definimos la función:

$$\phi(i, isNewYork) = \begin{cases} \text{costo mínimo de un plan válido para los primeros } i \text{ meses,} \\ \text{terminando en New York, si } isNewYork = \text{true}, \\ \text{terminando en San Francisco, si } isNewYork = \text{false}. \end{cases}$$

Para memorizar los resultados se usará una tabla:

$$mem[1..n][0..1],$$

donde:

- $mem[i][1]$ almacenará el valor de $\phi(i, \text{true})$;
- $mem[i][0]$ almacenará el valor de $\phi(i, \text{false})$.

Reformulación de la salida con la función objetivo

Una vez definidos los estados, el costo óptimo buscado es:

$$OPT[n] = \min\{\phi(n, \text{true}), \phi(n, \text{false})\}.$$

Planteamiento recurrente

Para el mes 1, no existe costo de mudanza previo. Por tanto:

$$\phi(1, \text{true}) = NY[1], \quad \phi(1, \text{false}) = SF[1].$$

Ahora, si $i \geq 2$ y el plan termina en New York, entonces existen dos posibilidades:

- el mes $i - 1$ también terminó en New York;
- el mes $i - 1$ terminó en San Francisco y se paga una mudanza.

Por ello:

$$\phi(i, \text{true}) = NY[i] + \min\{\phi(i - 1, \text{true}), \phi(i - 1, \text{false}) + M\}.$$

De manera análoga, si el plan termina en San Francisco:

$$\phi(i, \text{false}) = SF[i] + \min\{\phi(i - 1, \text{false}), \phi(i - 1, \text{true}) + M\}.$$

Descripción del algoritmo en castellano

El algoritmo resuelve el problema mediante una función recursiva $\phi(i, isNewYork)$. Esta función devuelve el costo mínimo para los primeros i meses, bajo la condición de que el plan termine el mes i en la ciudad indicada por el booleano.

Cada vez que la función es llamada:

- si el valor ya fue calculado antes, se retorna inmediatamente desde la tabla de memoria;
- en caso contrario, se calcula usando la recurrencia correspondiente, se guarda en memoria y luego se retorna.

Finalmente, como todo plan óptimo de longitud n debe terminar o bien en New York o bien en San Francisco, la respuesta final es el mínimo entre ambas posibilidades.

Pseudocódigo

```
phi(i, isNewYork) :  
    b = 1 si isNewYork = true, y 0 en otro caso  
  
    if mem[i][b] = -1 then  
        if i = 1 then  
            if isNewYork then ans = NY[1]  
            else ans = SF[1]  
        else  
            if isNewYork then  
                ans = NY[i] + mín(phi(i - 1, true), phi(i - 1, false) + M)  
            else  
                ans = SF[i] + mín(phi(i - 1, false), phi(i - 1, true) + M)  
            mem[i][b] = ans  
  
    ans = mem[i][b]  
    return ans  
  
SOLVE(NY, SF, n, M) :  
    for i = 1 to n do  
        mem[i][0] = -1  
        mem[i][1] = -1  
  
    OPT[n] = mín(phi(n, true), phi(n, false))  
    return OPT[n]
```

Ejemplo de funcionamiento

Considere el ejemplo del enunciado con:

$$n = 4, \quad M = 10,$$

y costos

	Mes 1	Mes 2	Mes 3	Mes 4
<i>NY</i>	1	3	20	30
<i>SF</i>	50	20	2	4

Entonces:

$$\phi(1, \mathbf{true}) = 1, \quad \phi(1, \mathbf{false}) = 50.$$

Para el mes 2:

$$\begin{aligned}\phi(2, \mathbf{true}) &= 3 + \text{mín}\{1, 50 + 10\} = 4, \\ \phi(2, \mathbf{false}) &= 20 + \text{mín}\{50, 1 + 10\} = 31.\end{aligned}$$

Para el mes 3:

$$\phi(3, \text{true}) = 20 + \min\{4, 31 + 10\} = 24,$$

$$\phi(3, \text{false}) = 2 + \min\{31, 4 + 10\} = 16.$$

Para el mes 4:

$$\phi(4, \text{true}) = 30 + \min\{24, 16 + 10\} = 54,$$

$$\phi(4, \text{false}) = 4 + \min\{16, 24 + 10\} = 20.$$

Por lo tanto:

$$OPT[4] = \min\{54, 20\} = 20.$$

Análisis de solapamiento de subproblemas

Si se intentara resolver el problema mediante recursión pura, al calcular un estado como

$$\phi(i, \text{true}),$$

sería necesario volver a calcular los estados

$$\phi(i-1, \text{true}) \quad \text{y} \quad \phi(i-1, \text{false}),$$

y estos mismos estados también reaparecen al calcular

$$\phi(i, \text{false}).$$

Por tanto, los mismos subproblemas se reutilizan varias veces. La memorización evita esta recomputación, ya que cada estado

$$(i, isNewYork)$$

se resuelve a lo sumo una sola vez.

Análisis de complejidad

Sin aplicar Programación Dinámica:

Si se formula una solución recursiva sin memoria, en cada mes podrían explorarse dos posibilidades, dando lugar a un árbol de recursión exponencial. En el peor caso, la complejidad temporal sería:

$$O(2^n).$$

Aplicando Programación Dinámica con memorización:

El algoritmo tiene exactamente dos estados por cada mes:

$$\phi(i, \text{true}) \quad \text{y} \quad \phi(i, \text{false}).$$

Por lo tanto, el número total de estados es $2n$, y cada uno se calcula a lo sumo una sola vez. Como cada cálculo toma tiempo constante aparte de las llamadas recursivas ya memorizadas, el tiempo total es:

$$O(n).$$

Además, la tabla

$$mem[1..n][0..1]$$

usa espacio:

$$O(n).$$

Si además se considera la pila de recursión, el espacio total sigue siendo:

$$O(n).$$

Prueba de correctitud

Teorema. Para todo $i \in \{1, \dots, n\}$ y todo valor booleano $isNewYork$:

- si $isNewYork = \text{true}$, entonces $\phi(i, isNewYork)$ es el costo mínimo de un plan válido para los primeros i meses que termina en New York;
- si $isNewYork = \text{false}$, entonces $\phi(i, isNewYork)$ es el costo mínimo de un plan válido para los primeros i meses que termina en San Francisco.

En consecuencia,

$$OPT[n] = \min\{\phi(n, \text{true}), \phi(n, \text{false})\}$$

es el costo de un plan óptimo.

Demostración. Procederemos por inducción sobre el número de meses i .

Caso base. Para $i = 1$, si $isNewYork = \text{true}$, entonces el único plan posible que termina en New York en el mes 1 tiene costo:

$$\phi(1, \text{true}) = NY[1].$$

Análogamente, si $isNewYork = \text{false}$, el único plan posible que termina en San Francisco en el mes 1 tiene costo:

$$\phi(1, \text{false}) = SF[1].$$

Por lo tanto, ambos valores son correctos.

Paso inductivo. Supongamos como hipótesis inductiva que, para algún $i - 1 \geq 1$, los valores

$$\phi(i - 1, \text{true}) \quad \text{y} \quad \phi(i - 1, \text{false})$$

son correctos.

Queremos demostrar que también son correctos

$$\phi(i, \text{true}) \quad \text{y} \quad \phi(i, \text{false}).$$

Caso 1: $isNewYork = \text{true}$.

Si un plan válido termina en New York en el mes i , entonces necesariamente ocurre una de estas dos situaciones:

- el plan ya estaba en New York en el mes $i - 1$, en cuyo caso el costo total es

$$\phi(i - 1, \text{true}) + NY[i];$$

- el plan estaba en San Francisco en el mes $i - 1$ y se muda a New York, en cuyo caso el costo total es

$$\phi(i - 1, \text{false}) + M + NY[i].$$

Por hipótesis inductiva, ambos términos usan costos óptimos correctos para los primeros $i - 1$ meses. Por lo tanto, el mejor plan que termina en New York en el mes i tiene costo:

$$\phi(i, \text{true}) = NY[i] + \min\{\phi(i - 1, \text{true}), \phi(i - 1, \text{false}) + M\}.$$

Caso 2: $isNewYork = \text{false}$.

Si un plan válido termina en San Francisco en el mes i , entonces necesariamente ocurre una de estas dos situaciones:

- el plan ya estaba en San Francisco en el mes $i - 1$, en cuyo caso el costo total es

$$\phi(i - 1, \text{false}) + SF[i];$$

- el plan estaba en New York en el mes $i - 1$ y se muda a San Francisco, en cuyo caso el costo total es

$$\phi(i - 1, \text{true}) + M + SF[i].$$

Por hipótesis inductiva, ambos términos usan costos óptimos correctos para los primeros $i - 1$ meses. Por lo tanto, el mejor plan que termina en San Francisco en el mes i tiene costo:

$$\phi(i, \text{false}) = SF[i] + \min\{\phi(i - 1, \text{false}), \phi(i - 1, \text{true}) + M\}.$$

En ambos casos, el valor calculado por la recurrencia coincide con el costo mínimo correcto del subproblema.

Por el principio de inducción, la afirmación es válida para todo $i \in \{1, \dots, n\}$.

Corolario. Todo plan óptimo de longitud n debe terminar o bien en New York o bien en San Francisco. Como

$$\phi(n, \text{true})$$

es el mejor costo posible entre todos los planes que terminan en New York, y

$$\phi(n, \text{false})$$

es el mejor costo posible entre todos los planes que terminan en San Francisco, entonces

$$OPT[n] = \min\{\phi(n, \text{true}), \phi(n, \text{false})\}$$

es exactamente el costo de un plan óptimo.

1.2. Forward and backward substring (Ej. 3.8, Erickson, p. 128)

Resumen del problema

Se debe diseñar y analizar un algoritmo eficiente para encontrar la longitud de la subcadena contigua más larga que aparece tanto en sentido directo como en sentido inverso dentro de una cadena de entrada $A[0..n-1]$, bajo la condición de que ambas ocurrencias no se solapen.

Especificación del problema

Entrada:

Una cadena de texto

$$A[0..n-1].$$

Salida:

Un entero

$$ans \in \mathbb{N}$$

tal que ans representa la longitud de la subcadena contigua más larga de la cadena A cuya inversa también aparece en A , bajo la condición de que ambas ocurrencias no se solapen.

Se considera que una cadena

$$c_1 = c_{1,1}c_{1,2} \cdots c_{1,k}$$

es la inversa de otra cadena

$$c_2 = c_{2,1}c_{2,2} \cdots c_{2,k}$$

si y sólo si

$$c_{1,1} = c_{2,k}, c_{1,2} = c_{2,k-1}, \dots, c_{1,k} = c_{2,1}.$$

Estrategia de Programación Dinámica a utilizar

Se utilizará **programación dinámica con memorización (top-down)**. La idea es definir una función recursiva

$$\phi(i, j, v),$$

donde:

- i es el índice izquierdo de la subcadena actual;
- j es el índice derecho de la subcadena actual;
- v es un valor booleano que indica si ya se inició la construcción de una subcadena candidata.

La elección de esta estrategia se justifica porque:

- el problema tiene **subestructura óptima**, pues la mejor solución en $A[i..j]$ depende de soluciones óptimas en subintervalos más pequeños;
- existen **subproblemas superpuestos**, ya que los mismos estados (i, j, v) aparecen repetidamente en distintos caminos recursivos.

Planteamiento de la función objetivo

Definimos:

$$\phi(i, j, v)$$

como la longitud de la subcadena contigua más larga c_1 tal que c_1 y su inversa c_2 ocurren dentro del segmento

$$A[i..j],$$

bajo la condición de que ambas ocurrencias no se solapen, y donde:

- si $v = \text{false}$, todavía no se ha iniciado la construcción de la subcadena candidata;
- si $v = \text{true}$, ya se inició la construcción y por tanto la continuidad debe mantenerse.

Para memorizar los resultados se usará una tabla:

$$mem[0..n-1][0..n-1][0..1],$$

donde:

- $mem[i][j][1]$ almacenará el valor de $\phi(i, j, \text{true})$;
- $mem[i][j][0]$ almacenará el valor de $\phi(i, j, \text{false})$.

Reformulación de la salida con la función objetivo

La salida final se obtiene evaluando la cadena completa y señalando que aún no se ha iniciado la construcción de una subcadena candidata:

$$ans = \phi(0, n-1, \text{false}).$$

Planteamiento recurrente

La recurrencia de $\phi(i, j, v)$ se define así:

$$\phi(i, j, v) = \begin{cases} 0, & \text{si } i \geq j, \\ 0, & \text{si } i < j \wedge A[i] \neq A[j] \wedge v = \text{true}, \\ 1 + \phi(i+1, j-1, \text{true}), & \text{si } i < j \wedge A[i] = A[j] \wedge v = \text{true}, \\ \max\{\phi(i+1, j, \text{false}), \phi(i, j-1, \text{false})\}, & \text{si } i < j \wedge A[i] \neq A[j] \wedge v = \text{false}, \\ \max\{1 + \phi(i+1, j-1, \text{true}), \phi(i+1, j, \text{false}), \phi(i, j-1, \text{false})\}, & \text{si } i < j \wedge A[i] = A[j] \wedge v = \text{false}. \end{cases}$$

La intuición es la siguiente:

- mientras no se haya iniciado la construcción ($v = \text{false}$), se puede intentar comenzar en los extremos si coinciden, o seguir buscando desplazando alguno de los dos índices;
- una vez iniciada la construcción ($v = \text{true}$), la continuidad obliga a que los siguientes caracteres también coincidan; si dejan de coincidir, la construcción termina.

Descripción del algoritmo en castellano

El algoritmo evalúa recursivamente subintervalos de la cadena usando la función $\phi(i, j, v)$. Si todavía no se ha iniciado una subcadena candidata, el algoritmo puede explorar tres posibilidades:

- comenzar la subcadena usando $A[i]$ y $A[j]$, si ambos caracteres coinciden;
- ignorar el carácter izquierdo y continuar buscando;
- ignorar el carácter derecho y continuar buscando.

En cambio, si la construcción ya fue iniciada, entonces la continuidad exige que los extremos actuales coincidan. Si no coinciden, la subcadena en construcción ya no puede seguir extendiéndose y el valor retornado es 0.

Cada vez que un estado (i, j, v) se calcula, su resultado se guarda en memoria para no volver a computarlo.

Pseudocódigo

```
phi( $i, j, v$ ) :  
     $b = 1$  si  $v = \text{true}$ , y 0 en otro caso  
  
    if  $mem[i][j][b] = -1$  then  
        if  $i \geq j$  then  
             $ans = 0$   
        else if  $v = \text{true}$  then  
            if  $A[i] = A[j]$  then  $ans = 1 + \text{phi}(i + 1, j - 1, \text{true})$   
            else  $ans = 0$   
        else  
            if  $A[i] = A[j]$  then  
                 $ans = \text{máx}(1 + \text{phi}(i + 1, j - 1, \text{true}),$   
                     $\text{phi}(i + 1, j, \text{false}), \text{phi}(i, j - 1, \text{false}))$   
            else  
                 $ans = \text{máx}(\text{phi}(i + 1, j, \text{false}), \text{phi}(i, j - 1, \text{false}))$   
             $mem[i][j][b] = ans$   
  
     $ans = mem[i][j][b]$   
    return  $ans$   
  
SOLVE( $A, n$ ) :  
    for  $i = 0$  to  $n - 1$  do  
        for  $j = 0$  to  $n - 1$  do  
             $mem[i][j][0] = -1$   
             $mem[i][j][1] = -1$   
  
     $ans = \text{phi}(0, n - 1, \text{false})$   
    return  $ans$ 
```


Ejemplo de funcionamiento

Considere la cadena:

$$A = \text{REDIVIDE}.$$

El algoritmo inicia evaluando:

$$\phi(0, 7, \text{false}).$$

Como todavía no ha iniciado la construcción, puede intentar empezar una subcadena candidata cuando encuentre extremos compatibles o, en caso contrario, seguir buscando en subintervalos más pequeños. En este ejemplo, la mejor coincidencia válida corresponde a la subcadena

$$\text{EDI},$$

cuya longitud es

$$3.$$

Por lo tanto, el algoritmo retorna:

$$ans = 3.$$

Análisis de solapamiento de subproblemas

La función $\phi(i, j, v)$ depende únicamente de los parámetros

$$(i, j, v).$$

Esto implica que diferentes caminos recursivos pueden llegar exactamente al mismo estado.

Por ejemplo, al expandir

$$\phi(0, 7, \text{false}),$$

pueden aparecer llamadas como

$$\phi(1, 7, \text{false}) \quad \text{y} \quad \phi(0, 6, \text{false}),$$

y a su vez ambas pueden generar el mismo subproblema

$$\phi(1, 6, \text{false}).$$

Por tanto, existe un claro solapamiento de subproblemas, lo que justifica almacenar las respuestas parciales mediante memorización.

Análisis de complejidad

Sin aplicar Programación Dinámica:

Si el algoritmo se implementa únicamente de forma recursiva, cada estado puede generar hasta máximo tres llamadas adicionales, lo que conduce a un árbol de recursión exponencial. En el peor caso, la complejidad temporal es:

$$O(3^n).$$

Aplicando Programación Dinámica con memorización:

El valor de $\phi(i, j, v)$ depende únicamente de:

- el índice i ;
- el índice j ;
- el booleano v .

El número total de estados posibles es:

$$O(n^2),$$

pues i y j toman a lo sumo n valores cada uno, y v tiene únicamente dos posibilidades.

Cada estado se calcula a lo sumo una sola vez y cada cálculo cuesta tiempo constante aparte de las llamadas recursivas ya memorizadas. Por ello, la complejidad temporal total es:

$$O(n^2).$$

La memoria usada por la tabla

$$mem[0..n-1][0..n-1][0..1]$$

también es:

$$O(n^2).$$

Si además se considera la pila de recursión, el espacio total sigue estando acotado por:

$$O(n^2).$$

Prueba de correctitud

Teorema

Para todo par de índices $0 \leq i \leq j < n$ y todo valor booleano $v \in \{\mathbf{true}, \mathbf{false}\}$, la función

$$\phi(i, j, v)$$

retorna correctamente la longitud de la subcadena contigua más larga contenida en $A[i..j]$ cuya inversa también aparece en $A[i..j]$, bajo la condición de que ambas ocurrencias no se solapen, interpretando v como el estado de construcción de la subcadena candidata.

En consecuencia,

$$ans = \phi(0, n-1, \mathbf{false})$$

es la respuesta correcta del problema.

Demostración

Procederemos por inducción sobre la longitud del intervalo

$$L = j - i + 1.$$

Caso base

Si

$$i \geq j,$$

entonces el intervalo tiene longitud 0 o 1, y no es posible construir dentro de él una pareja válida de ocurrencias no solapadas en sentido directo e inverso. Por tanto:

$$\phi(i, j, v) = 0.$$

Luego el caso base es correcto.

Además, si

$$i < j, \quad A[i] \neq A[j], \quad v = \text{true},$$

entonces la construcción ya había sido iniciada, pero los caracteres actuales no coinciden, así que la continuidad de la subcadena candidata se rompe. En ese caso también es correcto retornar:

$$\phi(i, j, \text{true}) = 0.$$

Paso inductivo

Supongamos como hipótesis inductiva que para todo intervalo de longitud estrictamente menor que L , la función ϕ retorna el valor correcto. Queremos demostrar que entonces también lo hace para un intervalo $A[i..j]$ de longitud L .

Caso 1: $v = \text{true}$.

Si la construcción ya fue iniciada, la continuidad obliga a que los extremos actuales coincidan.

- Si $A[i] = A[j]$, entonces se puede extender la subcadena candidata en una unidad y continuar resolviendo el subproblema interno:

$$\phi(i, j, \text{true}) = 1 + \phi(i + 1, j - 1, \text{true}).$$

Por hipótesis inductiva, el valor del subproblema interno es correcto, así que el resultado total también lo es.

- Si $A[i] \neq A[j]$, la continuidad se rompe y no es posible extender más la subcadena candidata, por lo que:

$$\phi(i, j, \text{true}) = 0.$$

Esto también es correcto.

Caso 2: $v = \text{false}$.

Si todavía no se ha iniciado la construcción, entonces la mejor respuesta en $A[i..j]$ debe surgir de una de las siguientes opciones:

- empezar la subcadena usando los extremos, si $A[i] = A[j]$, lo que produce

$$1 + \phi(i + 1, j - 1, \text{true});$$

- descartar el extremo izquierdo y buscar la mejor solución en

$$A[i + 1..j],$$

es decir,

$$\phi(i + 1, j, \text{false});$$

- descartar el extremo derecho y buscar la mejor solución en

$$A[i..j - 1],$$

es decir,

$$\phi(i, j - 1, \text{false}).$$

Por hipótesis inductiva, cada uno de esos subproblemas está correctamente resuelto. Como toda solución válida en $A[i..j]$ necesariamente cae en una de esas categorías, tomar el máximo entre ellas produce exactamente la mejor respuesta posible para el intervalo completo.

Por lo tanto, $\phi(i, j, \text{false})$ también es correcto.

Corolario

Aplicando el teorema al intervalo completo de la cadena, se concluye que

$$\phi(0, n - 1, \text{false})$$

retorna la longitud de la subcadena contigua más larga de A cuya inversa también aparece en A , sin solapamiento entre ambas ocurrencias. En consecuencia,

$$ans = \phi(0, n - 1, \text{false})$$

es la salida correcta del problema.

2. Problemas prácticos (entregados en la arena)

Archivos a entregar:

- `cat.py`
- `business.py`
- `duathlon.py`
- `nod.py`
- `pair.py`